Gebze Technical University

CSE 341 — Concepts of Programming Languages - Spring 2026

Project Part 2 — Retrospective (D5)

ScoreScript

Student Number: **230104004090**
Name: **FATİH EMRE OĞAN**
Submission Date: 22 May 2026

What is Working

ScoreScript Part 2 is fully implemented and working end-to-end. The complete pipeline runs as follows: the lexer tokenises the source, the parser builds the AST, the type checker validates all type rules statically and the interpreter executes the program. All three valid example programs run correctly and produce accurate output.

The type checker enforces all eight rules specified in the design:

- Undefined variable, function, student or group references are rejected
- Type mismatches in declarations and assignments are caught
- void function results used as values are rejected
- Non-bool conditions in if and for statements are rejected
- curve, average and rank reject non-group arguments
- transcript rejects non-student arguments
- Function call arity and argument types are validated
- No implicit coercion between int and float

The interpreter correctly executes all four domain operations: average computes real averages, curve applies the [0,100] clamp, rank sorts students by average descending, and transcript prints formatted student records. Control flow (if/else, for loops), variable declarations, assignments, arithmetic, comparisons, logical operators, and user-defined functions with return values all work correctly.

Two bugs were found and fixed during Part 2 testing: the float literal mismatch in program2.ss (avg >= 90 compared to float, which violated the strict no-coercion rule) and a Java locale bug in Interpreter.java where %.2f printed 82,33 on Turkish-locale Windows instead of 82.33. Both were caught through testing and fixed before submission.

What Was Hard

The hardest part of Part 2 was writing the operational semantics in D1. I had never written formal inference rules before and understanding the big-step notation from Sebesta took me a while. Getting the state model right especially showing that average does not mutate sigma while curve does required several attempts. On the implementation side, understanding how the scope stack should work for static scoping was tricky at first. I had to make sure that when a function executes, it sees the global scope and not the caller's local variables. The locale bug was also unexpected I did not realize that Java's printf is locale-sensitive until I saw commas in the output on my own machine.

What Changed from the Part 1 Plan

In the Part 1 retrospective, I planned to add a type checker and interpreter in Part 2, complete the remaining D1 sections and execute the four domain operations end-to-end. All of these were completed as planned.

Two things were not anticipated in the Part 1 plan. First, the strict no-coercion rule required updating program2.ss the float literals in letterGrade had to be changed from 90 to 90.0 to satisfy the type checker. This was actually a good outcome: it confirmed the type checker was working correctly and enforced a rule that was already in D1. Second, a Java locale bug in Interpreter.java caused decimal numbers to print with a comma (82,33) instead of a dot (82.33). This happened because Java's printf is locale-sensitive and my Windows machine uses Turkish regional settings where the decimal separator is a comma. This was found during E3 testing and fixed with Locale.ROOT, which also improved D1-D2 consistency since ScoreScript's own float syntax uses a dot.

Self-Assessment

The table below gives my honest assessment of each deliverable against the project rubric. The grade column reflects what I believe the work merits; the justification column is where I explain my reasoning.

Criterion	Grade	Justification
D1 — Design Specification	**B**	All eight sections are complete and the Sebesta references are correct. The operational semantics section in §4.4 required the most effort it was my first time writing formal inference rules and getting the state model right took several attempts. I am giving myself a B because I want to be honest that this section pushed me the most .
D2 — Implementation	**A**	The type checker and interpreter both work correctly on all test programs. Two bugs were found during testing and fixed before submission. The implementation is fully consistent with the D1 specification.
D3 — Test Report	**A**	All three valid programs show real interpreter output with correct numbers verified on my own machine. The type error program and five parse error programs are all documented with actual terminal output.
D4 — AI Usage Journal	**B**	The major AI interactions are documented with prompts, what was accepted, what was rejected, and errors caught. E2 and E3 experiments are included with genuine test results. Some reflection paragraphs could go deeper on why certain AI suggestions were wrong.
D5 — Retrospective	**A**	The retrospective honestly covers what worked, what was hard, what was unexpected and includes a detailed self-assessment with justification for every criterion.
Overall Language Design	**B**	ScoreScript is a coherent DSL with a working type checker and interpreter. The grammar is unambiguous and the domain operations work correctly. A full A would require more language features and deeper test coverage, including runtime error cases that the current grammar cannot express.